

Crear la instancia "CMS" / Entrar en la instancia

...

Instalación de docker en un comando:

```
#!/bin/bash
```

```
# Actualizar e instalar dependencias
```

```
sudo apt update && \
```

```
sudo apt install -y ca-certificates curl && \
```

```
sudo install -m 0755 -d /etc/apt/keyrings && \
```

```
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg -o /etc/apt/keyrings/docker.asc  
&& \
```

```
sudo chmod a+r /etc/apt/keyrings/docker.asc && \
```

```
# Añadir repositorio oficial de Docker
```

```
sudo tee /etc/apt/sources.list.d/docker.sources > /dev/null <<EOF
```

```
Types: deb
```

```
URIs: https://download.docker.com/linux/ubuntu
```

```
Suites: $(. /etc/os-release && echo "${UBUNTU_CODENAME:-$VERSION_CODENAME}")
```

```
Components: stable
```

```
Signed-By: /etc/apt/keyrings/docker.asc
```

```
EOF
```

```
# Instalar Docker
```

```
sudo apt update && \
```

```
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
```

```
docker-compose-plugin && \
```

```
# Añadir usuario al grupo docker
```

```
sudo usermod -aG docker ${USER}
```

```
echo " 🚀 Docker instalado. Cierra sesión y vuelve a entrar para aplicar los cambios."
```

Creo un contenedor con wordpress

```
docker run -it --name wordpress -p 1080:80 ubuntu:24.04 /bin/bash
```

Instalo apache dentro del contenedor wordpress

```
apt install apache2
```

Instalo mariadb dentro del contenedor wordpress

```
apt install mariadb-server
```

Instalo php dentro del contenedor wordpress

```
apt install php php-mysql php-xml php-gd php-curl php-mbstring php-zip
```

Instalamos wget y usamos wget para descargar wordpress en un zip

```
apt install wget  
cd /var/www/html/  
wget https://wordpress.org/latest.zip
```

Instalamos unzip y lo usamos para descomprimir el wordpress

```
apt install unzip  
unzip latest.zip
```

Borramos el zip y el html que ya no nos sirve

```
rm latest.zip  
rm index.html
```

Movemos todos los archivos dentro de la carpeta wordpress y lo sacamos

```
mv wordpress/* /var/www/html/
```

Ahora vamos a crear la base de datos (cambia tu nombre):

```
service mariadb start
```

```
mariadb
```

```
CREATE DATABASE wordpress_db;
```

```
CREATE USER 'wp_antonio'@'localhost' IDENTIFIED BY 'antonio1234';
```

```
GRANT ALL PRIVILEGES ON wordpress_db.* TO 'wp_antonio'@'localhost';
```

```
FLUSH PRIVILEGES;
```

```
EXIT;
```

Copiamos el archivo de ejemplo “wp-config-sample.php” en la ruta /var/www/html y le llamamos a la copia “wp-config.php” porque ese es el archivo que wordpress buscará con nuestros datos

```
cp wp-config-sample.php wp-config.php
```

Instalamos vim y configuramos el archivo

```
vim wp-config.php
```

Pon los datos que acabas de poner en tu base de datos

```
define('DB_NAME', 'wordpress_db'); // nombre de la base de datos
define('DB_USER', 'wp_user'); // usuario de la base de datos
define('DB_PASSWORD', 'tu_contraseña_segura'); // contraseña del usuario
```

(Cuando hagas los cambios guardas y sales del archivo pulsando Escape y escribes :wq)

Añadir regla de entrada en la instancia con puerto 1080 en AWS:

TCP personalizado ▼	TCP	1080	Anywhe... ▼	Q 0.0.0.0/0		Eliminar
				0.0.0.0/0 X		

De paso añadimos las reglas de entrada de los otros 2 gestores de contenido (pondremos mismamente la 1180 y 1280)

TCP personalizado ▼	TCP	1080	Anywhe... ▼	Q 0.0.0.0/0		Eliminar
				0.0.0.0/0 X		
TCP personalizado ▼	TCP	1180	Anywhe... ▼	Q 0.0.0.0/0		Eliminar
				0.0.0.0/0 X		
TCP personalizado ▼	TCP	1280	Anywhe... ▼	Q 0.0.0.0/0		Eliminar
				0.0.0.0/0 X		

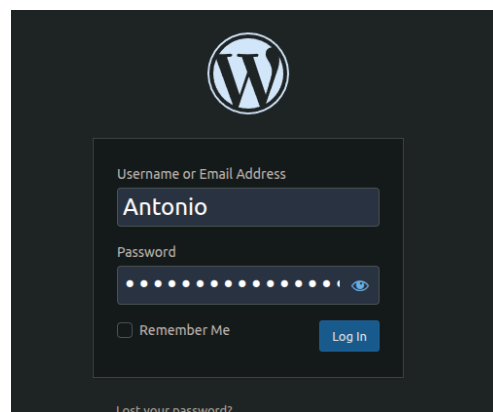
Iniciamos los servicios

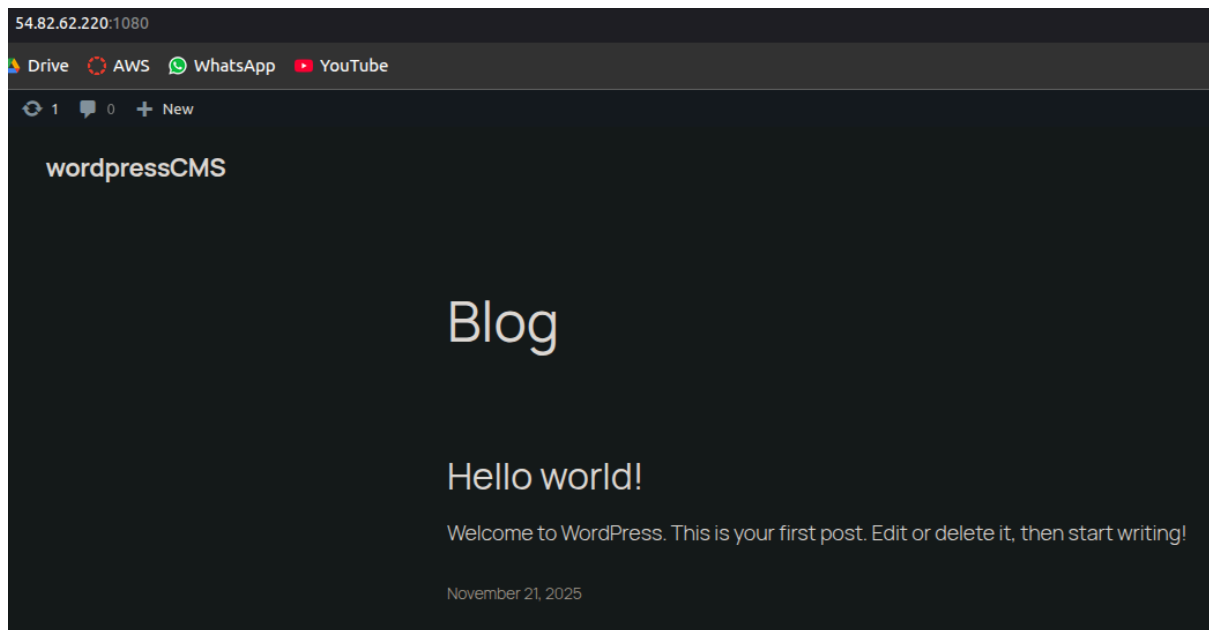
```
service mariadb start
service apache2 start
```

Ahora simplemente copias tu ip pública de la instancia, la pegas y pones el puerto (en el navegador). Luego rellenas los datos e instalas.

54.82.62.220:1080 (en mi caso)

Finalmente te logeas y buscas en el navegador la misma ip con puerto



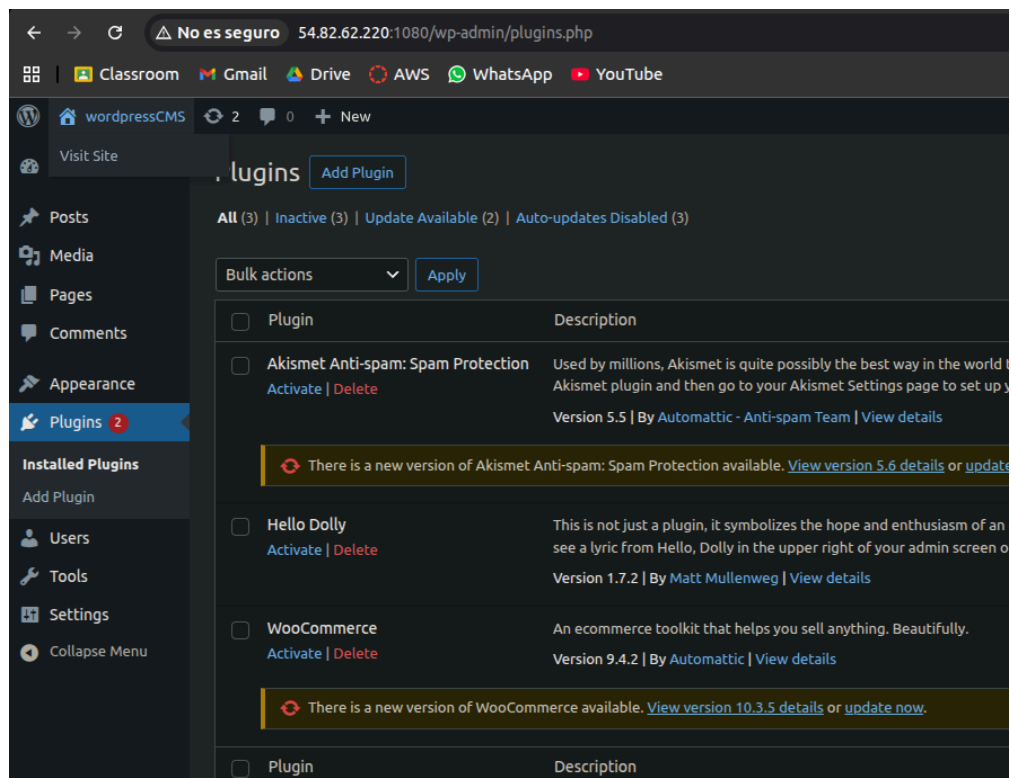


Instalamos el plugin de WooCommerce

```
cd /var/www/html/wp-content/plugins
wget https://downloads.wordpress.org/plugin/woocommerce.9.4.2.zip
unzip woocommerce.9.4.2.zip
rm woocommerce.9.4.2.zip
```

Accedemos a plugins y hacemos capturas de pantalla del plugin WooCommerce:

(Le damos a activate en WooCommerce)



Crear imagen con apache / mariadb / php (Github)

Desde la instancia creamos el siguiente docker:

```
docker run -it --name CMS ubuntu:24.04 /bin/bash
```

```
apt update
```

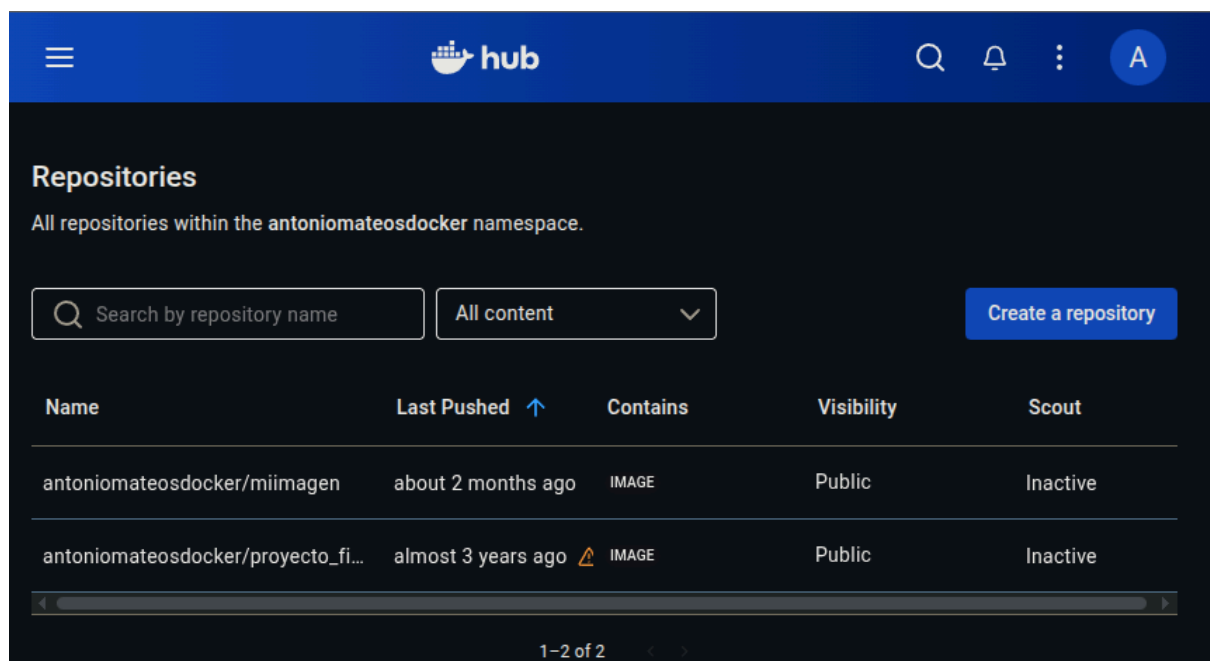
```
apt install apache2  
service apache2 start
```

```
apt install mariadb-server  
service mariadb status  
service mariadb start
```

```
apt install php php-mysql php-xml php-gd php-curl php-mbstring php-zip
```

```
apt install wget  
apt install vim  
apt install unzip
```

Vamos a docker hub



Hacemos “docker login” en la instancia de amazon (fuera de contenedores)

Y pinchamos en el link en el que pondremos el código que nos da

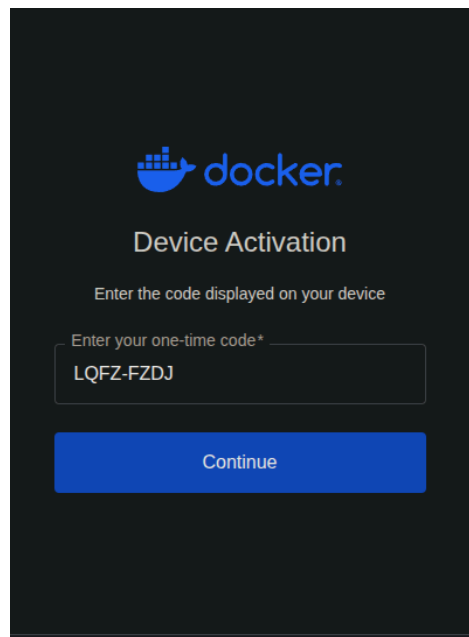
```
ubuntu@ip-172-31-19-39:~$ docker login

USING WEB-BASED LOGIN

Info → To sign in with credentials on the command line, use 'docker login -u <username>'

Your one-time device confirmation code is: LQFZ-FZDJ
Press ENTER to open your browser or submit your device code here: https://login.docker.com/activate

Waiting for authentication in the browser...
```



Creamos una imagen de nuestro contenedor

(antonimateosdocker es usuario de docker hub)

`docker commit <nombreContenedor> <nombreContenedor>:<tag>`

`docker commit CMS cms:web`

Ahora hacemos el tag de la imagen y luego el push para subirlo

```
ubuntu@ip-172-31-19-39:~$ docker images
INFO → U In Use
IMAGE          ID                DISK USAGE  CONTENT SIZE  EXTRA
cms:web        683976cc94bd      827MB       175MB
ubuntu:24.04   c35e29c94501      117MB       29.7MB      U
ubuntu@ip-172-31-19-39:~$ docker tag cms:web antonimateosdocker/cms:web
ubuntu@ip-172-31-19-39:~$ docker push antonimateosdocker/cms:web
The push refers to repository [docker.io/antonimateosdocker/cms]
c0d3639d2f57: Pushed
20043066d3d5: Mounted from library/ubuntu
web: digest: sha256:683976cc94bd59ccacdf1fba1e60e2ee9594c2c769f8340ce3ad332ee20973cc size: 752
ubuntu@ip-172-31-19-39:~$
```

`docker tag cms:web antonimateosdocker/cms:web`

`docker push antonimateosdocker/cms:web`

Crear contenedor docker para el gestor de contenidos “Drupal” y hacemos update

```
docker run -it --name Drupal -p 1180:80 cms:web /bin/bash
```

```
service apache2 start
```

```
service mariadb start
```

```
mariadb (reemplaza mi nombre por el tuyo)
```

Creamos base de datos:

```
CREATE DATABASE drupal_db;
```

```
CREATE USER 'drupal_antonio'@'localhost' IDENTIFIED BY 'antonio1234';
```

```
GRANT ALL PRIVILEGES ON drupal_db.* TO 'drupal_antonio'@'localhost';
```

```
FLUSH PRIVILEGES;
```

```
EXIT;
```

```
cd /var/www/html
```

```
rm index.html
```

```
wget https://www.drupal.org/download-latest/zip -O drupal.zip
```

```
unzip drupal.zip
```

```
mv drupal-11.2.8/* /var/www/html/
```

```
chown -R www-data:www-data /var/www/html/
```

```
chmod -R 755 /var/www/html/
```

Creamos el siguiente archivo:

```
vim /etc/apache2/sites-available/example.com.conf
```

Pegamos esto:

```
<VirtualHost *:80>
```

```
    ServerName example.com
```

```
    ServerAdmin admin@example.com
```

```
    DocumentRoot /var/www/html
```

```
<Directory /var/www/html>
```

```
    Options FollowSymlinks
```

```
    AllowOverride All
```

```
    Require all granted
```

```
</Directory>
```

```
    ErrorLog ${APACHE_LOG_DIR}/error.log
```

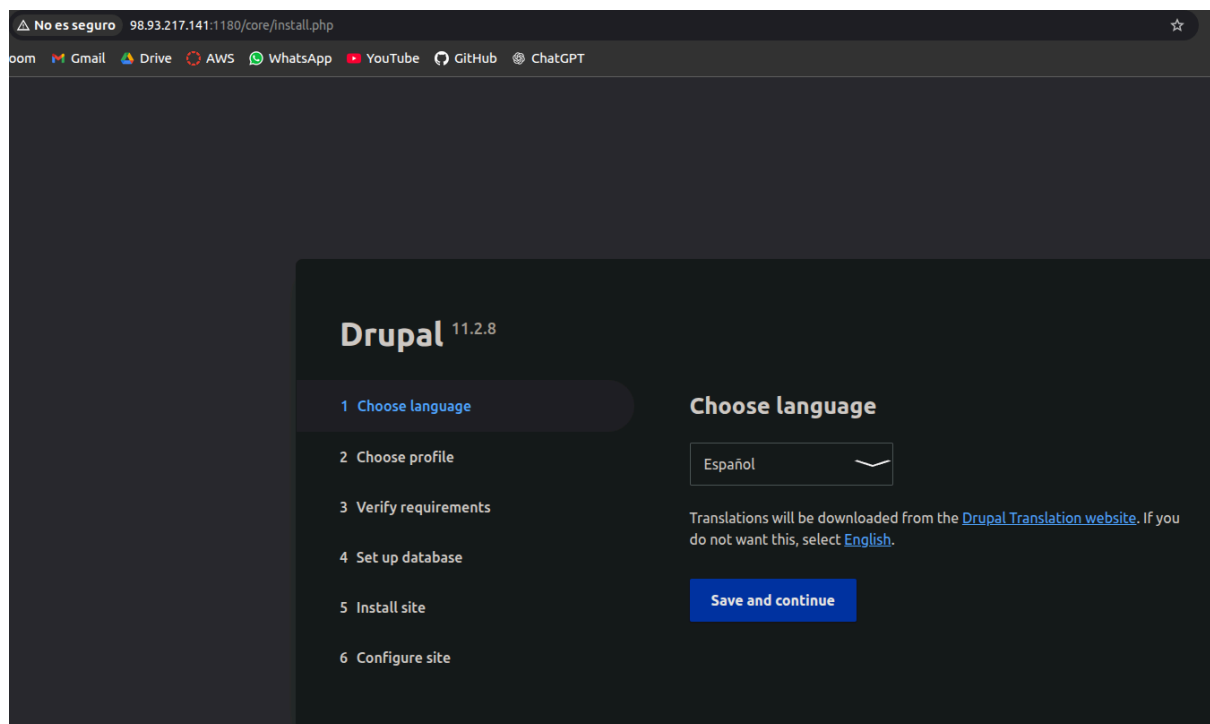
```
    CustomLog ${APACHE_LOG_DIR}/access.log combined
```

```
<Directory /var/www/html>
  RewriteEngine on
  RewriteBase /
  RewriteCond %{REQUEST_FILENAME} !-f
  RewriteRule ^(.*) index.php [PT,L]
</Directory>
</VirtualHost>
```

(Sales y guardas con “:wq!”)

```
a2ensite example.com
a2enmod rewrite
service apache2 restart
a2enmod rewrite
```

Desplegamos en el navegador con nuestra ip pública y la ip por la que escucha apache



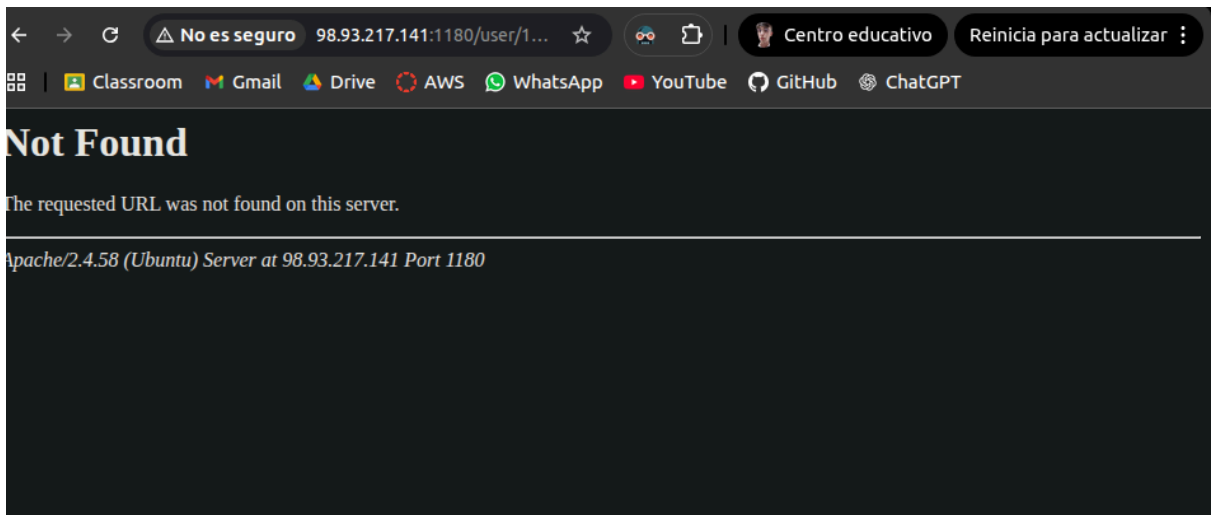
Completas todo lo que te pida y cuando llegues al 3 si no tienes ningún warning vas abajo del todo a “avanzar de todos modos”

Pones los datos de la base de datos sin tocar opciones avanzadas

The image shows the 'Configuración de la base de datos' (Database configuration) step in the Drupal 11.2.8 installation process. On the left, a vertical list of steps is shown: 1. Elegir un idioma, 2. Elegir perfil, 3. Verificar requisitos, 4. Configurar base de datos (highlighted in blue), 5. Instalar sitio, and 6. Configurar sitio. The main area contains the following fields and options:

- Tipo de base de datos ***: A radio button is selected for 'MySQL, MariaDB, Percona Server o equivalente'.
- Nombre de la base de datos**: A text input field containing 'drupal_db'.
- Nombre de usuario de la base de datos**: A text input field containing 'drupal_antonio'.
- Contraseña de la base de datos**: An empty password input field.
- Opciones avanzadas**: A dropdown menu currently showing a downward arrow.
- Guardar y continuar**: A blue button at the bottom right.

Cuando completes todos los datos te saldrá esto:



pones /index.php entre el puerto y el /user
ejemplo:

http://98.93.217.141:1180/index.php/user/1?check_logged_in=1

Cambiamos tipo de instancia a t3 medium para tener mas ram

Cambiar tipo de instancia

Información | Obtener asesoramiento

Puede cambiar el tipo de instancia solo si el tipo de instancia actual y el tipo de instancia que dese

ID de la instancia

 i-0bcdd80e717d92a8e (CMS)

Tipo de instancia actual

t3.micro

Nuevo tipo de instancia

 t3.medium 

Solo se pueden elegir tipos de instancia con x86 (32-bit) / (64-bit) arquitectura.

```
a2dissite 000-default.conf
a2ensite example.com.conf
a2enmod rewrite
service apache2 restart
```

PRESTASHOP:

```
docker run -it --name Prestashop -p 1280:80 cms:web
```

```
service apache2 start
service mariadb start

mariadb
```

```
CREATE DATABASE prestashop_db;
```

```
CREATE USER 'prestashop_antonio'@'localhost' IDENTIFIED BY 'antonio1234';
```

```
GRANT ALL PRIVILEGES ON prestashop_db.* TO 'prestashop_antonio'@'localhost';
```

```
FLUSH PRIVILEGES;
```

```
EXIT;
```

```
cd /var/www/html
```

```
wget
https://assets.prestashop3.com/dst/edition/corporate/9.0.0-1.0/prestashop_edition_classic_version_9.0.0-1.0.zip
```

```
unzip prestashop_edition_classic_version_9.0.0-1.0.zip
```

```
rm prestashop_edition_classic_version_9.0.0-1.0.zip
rm index.html
```

```
chown -R www-data:www-data /var/www/html
chmod -R 755 /var/www/html
```

```
apt update
apt install -y php-intl
service apache2 restart
```

```
a2enmod rewrite
service apache2 restart
```

```
vim /etc/apache2/sites-available/000-default.conf
```

Añade:

```
<Directory /var/www/html>
    AllowOverride All
    Require all granted
</Directory>
```

```
<VirtualHost *:80>
    # The ServerName directive sets the request scheme, hostname and port that
    # the server uses to identify itself. This is used when creating
    # redirection URLs. In the context of virtual hosts, the ServerName
    # specifies what hostname must appear in the request's Host: header to
    # match this virtual host. For the default virtual host (this file) this
    # value is not decisive as it is used as a last resort host regardless.
    # However, you must set it for any further virtual host explicitly.
    #ServerName www.example.com

    ServerAdmin webmaster@localhost
    DocumentRoot /var/www/html

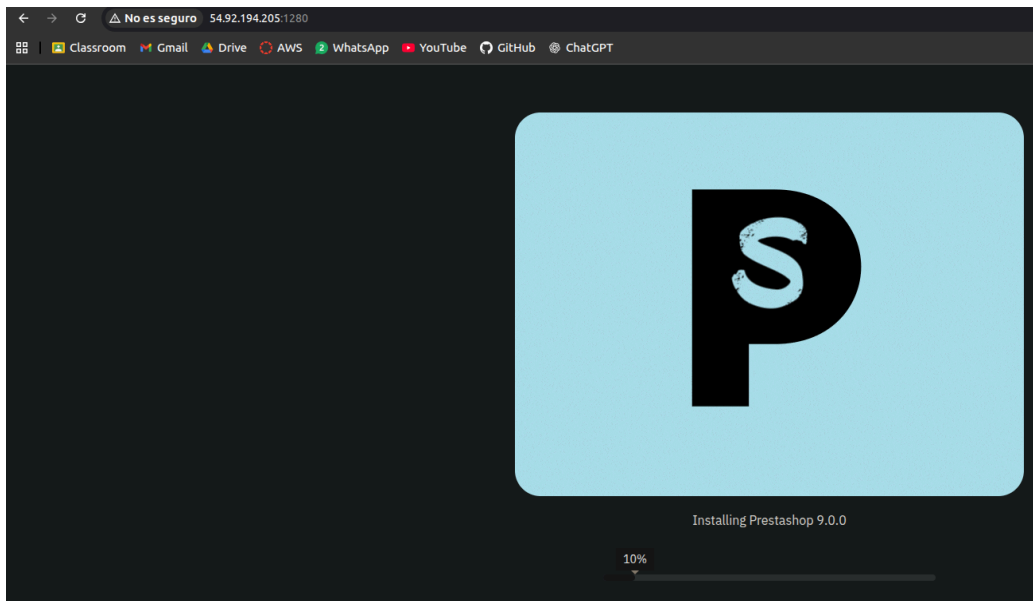
    # Available loglevels: trace8, ..., trace1, debug, info, notice, warn,
    # error, crit, alert, emerg.
    # It is also possible to configure the loglevel for particular
    # modules, e.g.
    #LogLevel info ssl:warn

    ErrorLog ${APACHE_LOG_DIR}/error.log
    CustomLog ${APACHE_LOG_DIR}/access.log combined

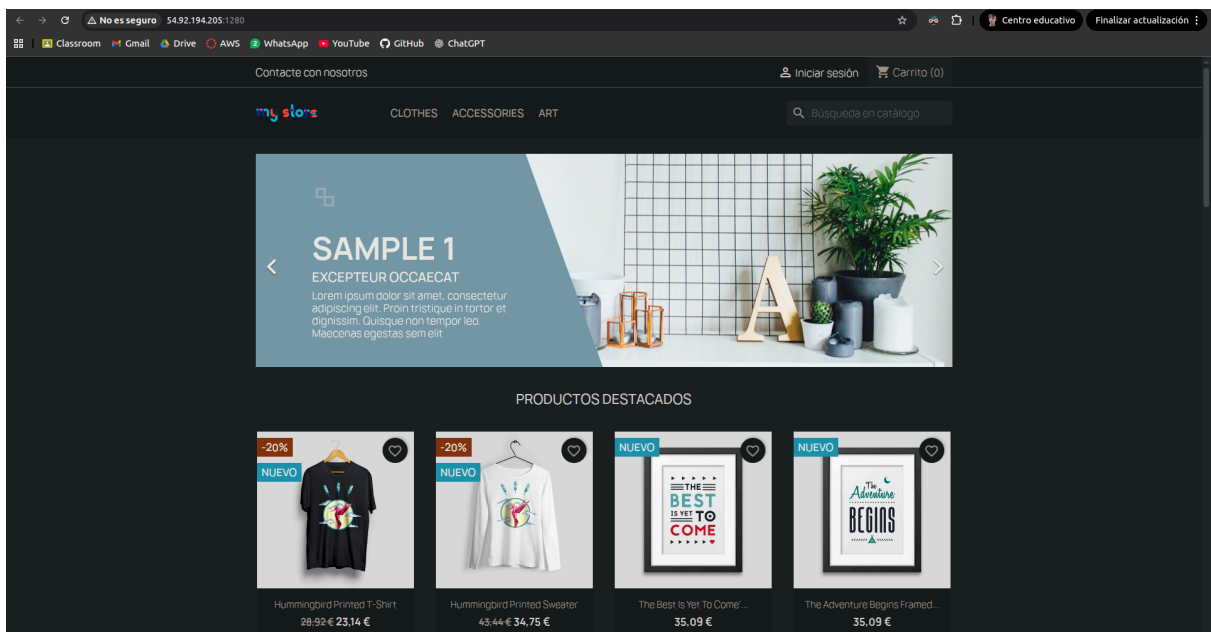
    # For most configuration files from conf-available/, which are
    # enabled or disabled at a global level, it is possible to
    # include a line for only one particular virtual host. For example the
    # following line enables the CGI configuration for this host only
    # after it has been globally disabled with "a2disconf".
    #Include conf-available/serve-cgi-bin.conf
    <Directory /var/www/html>
        AllowOverride All
        Require all granted
    </Directory>
</VirtualHost>
```

```
service apache2 restart
```

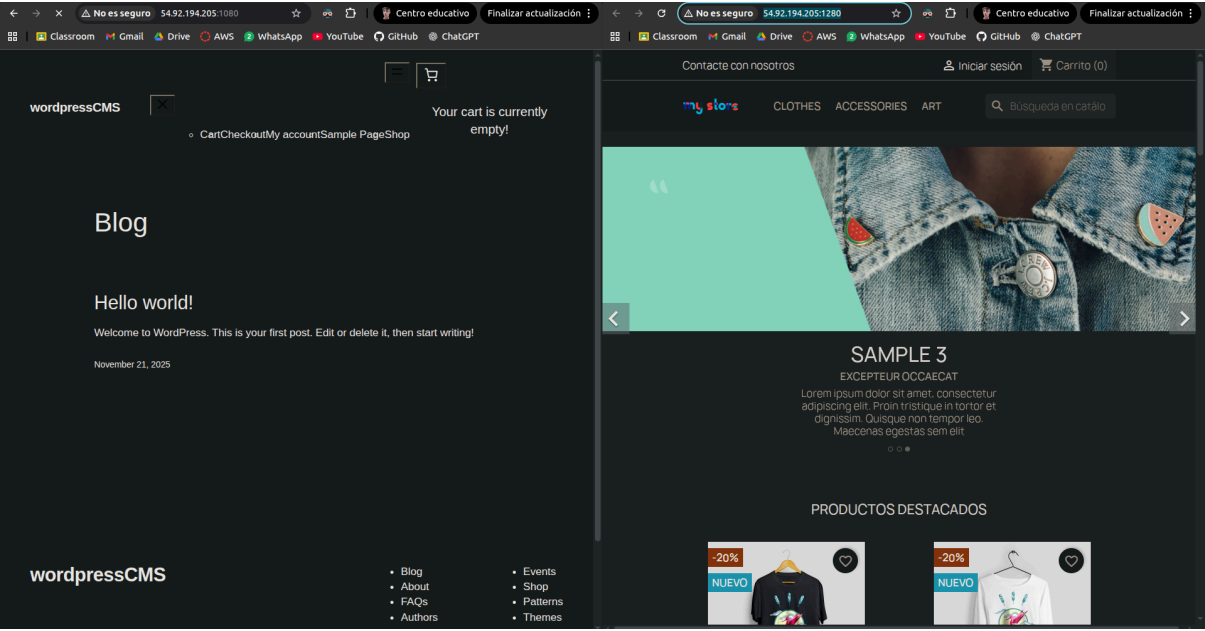
ipInstancia:1280



Una vez se instala:



Puedo desplegar wordpress y prestashop a la vez:



TODAS DESPLEGADAS:

